ER Diagram, Relational Schema & Tables

Smart City Utilities & Infrastructure Management System · DBMS Level 3 mini-project
Dhanush & Gagan · Design v1, 25 October 2026 · matches the approved plan (section D)

In simple words. PostgreSQL holds 15 tables: the 10 from the original design (kept with the same names, with their mistakes fixed) plus 5 new ones the original forgot: zones, login accounts, sessions, ticket history and a log of each monthly usage transfer from MongoDB. MongoDB holds the raw meter readings and flexible notes in 6 collections. This document covers rubric rows 1 (ER diagram, 2 marks) and 2 (relational schema with keys, relationships and constraints, 3 marks); the DDL in the appendix is the starting point for row 3.

1. Tables we will use (PostgreSQL)

#	Table	What it stores	Origin	Columns
1	`zone`	City → District → Ward hierarchy used for analytics and staff areas	**New**	4
2	`resident`	A citizen who owns properties (synthetic data only)	Original, modified	7
3	`property`	A house, shop or factory supplied with utilities	Original, modified	10
4	`utility_meter`	A physical electricity, water or gas meter on a property	Original, modified	7
5	`billing_cycle`	One monthly billing period with its due date and penalty	Original, modified	8
6	`utility_rate`	One tariff slab (tier) for a utility, valid for a date range	Original, modified	8
7	`aggregation_run`	One run of the MongoDB → PostgreSQL monthly usage bridge	**New**	8
8	`meter_reading`	Monthly consumption of one meter, aggregated from MongoDB (the bridge table)	Original, modified	10
9	`invoice`	The bill for one property in one billing cycle	Original, modified	8
10	`invoice_line_item`	One line of a bill: a utility × tier, or a late fee	Original, modified	9
11	`payment`	Every payment attempt (simulated gateway); at most one succeeds per invoice	Original, modified	9
12	`maintenance_ticket`	A repair job, opened by a resident, staff or the anomaly detector	Original, modified	16
13	`ticket_status_history`	Audit trail of every ticket status change (written by trigger)	**New**	6
14	`app_user`	A login account with its role (Resident, Staff or Admin)	**New**	8
15	`user_session`	A signed-in session; only a hash of the cookie token is stored	**New**	4

Changed from the original 10 tables: `government_id` becomes synthetic `citizen_ref`; `zone_type` text becomes a `zone` table; `utility_rate` allows an open-ended top slab and blocks overlapping tariffs; `meter_reading` stores opening and closing registers; `invoice` gets UNIQUE (property, cycle); line items record the slab and rate used; `payment` keeps every attempt with an idempotency key; tickets get assignment, de-duplication and history. Financial foreign keys use `ON DELETE RESTRICT` so history can never be erased.

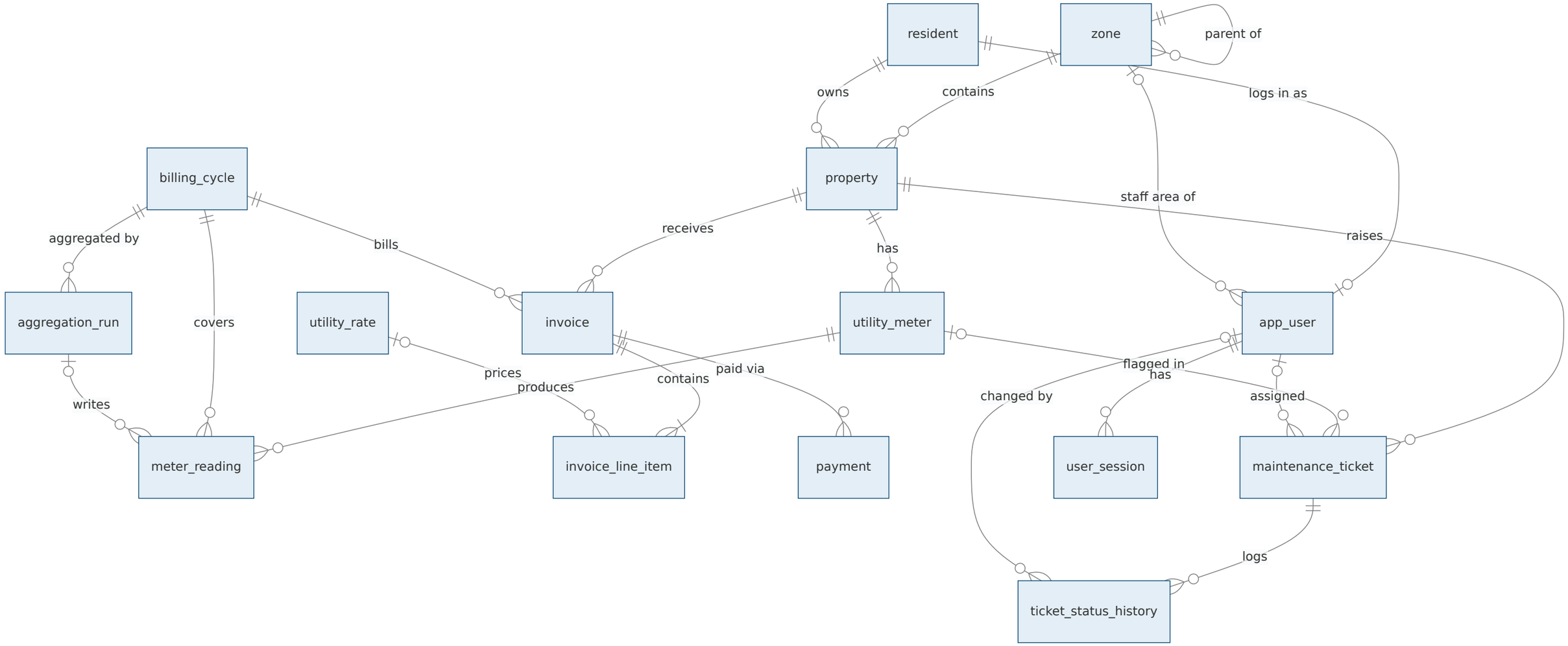
Collections we will use (MongoDB)

Collection	What it stores	Rules / indexes
`meter_readings`	Time-series collection: every raw smart-meter reading, with different fields per meter type	180-day TTL; index (meta.meter_code, ts)
`readings_daily`	Daily roll-up per meter, kept after raw readings expire	index (zone, day)
`anomaly_events`	Detected spikes, leaks and outages with the reason; also the outbox for creating SQL tickets	$jsonSchema validator; index (status, detected_at)
`maintenance_logs`	Free-form ticket notes, sensor snapshots and attachment metadata	$jsonSchema validator; unique (ticket_no)
`notifications`	In-app alerts for residents (unusual usage, due dates)	$jsonSchema validator; 30-day TTL
`ingestion_batches`	Simulator batch records so re-runs don't duplicate data	_id = batch id

The two databases are linked by codes, not foreign keys: MongoDB documents carry `meter_code`, `property_code` and `ticket_no`, which are UNIQUE columns in PostgreSQL.

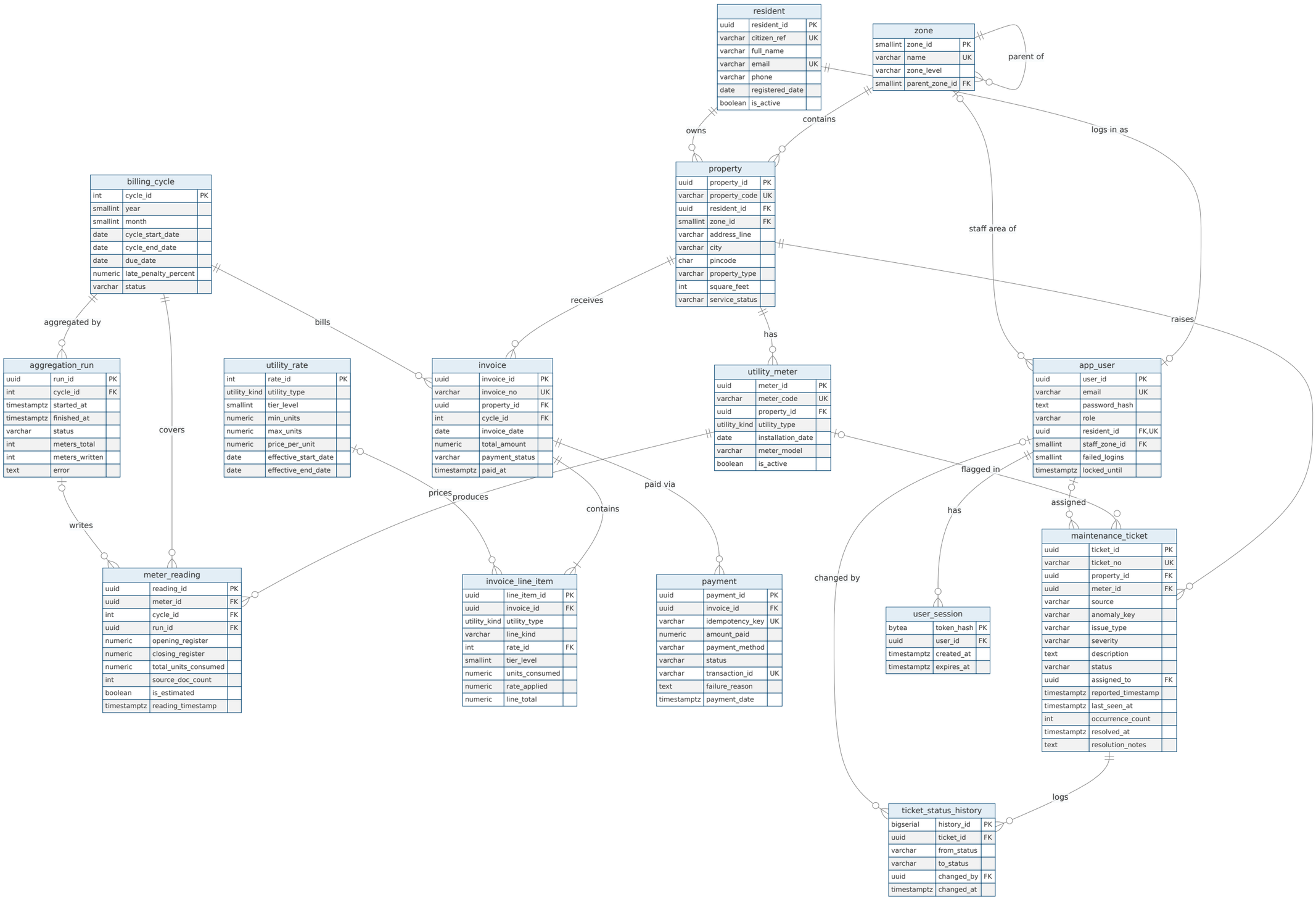
2. ER diagram: overview

Entities and relationships only, for reading at a glance. Crow's-foot notation: **||** exactly one, **|o** zero or one, **o{** zero or many, **|{** one or many.



3. ER diagram: full, with attributes

Every attribute with its type; PK = primary key, FK = foreign key, UK = unique key.



4. Relationships and cardinalities

Parent	Relationship	Child	Cardinality	Foreign key	ON DELETE
zone	parent of	zone	1 : M (self)	zone.parent_zone_id	RESTRICT
zone	contains	property	1 : M	property.zone_id	RESTRICT
resident	owns	property	1 : M	property.resident_id	RESTRICT
resident	logs in as	app_user	1 : 0..1	app_user.resident_id (UNIQUE)	RESTRICT
zone	staff area of	app_user	0..1 : M	app_user.staff_zone_id	NO ACTION
property	has	utility_meter	1 : M	utility_meter.property_id	RESTRICT
utility_meter	produces	meter_reading	1 : M	meter_reading.meter_id	RESTRICT
billing_cycle	covers	meter_reading	1 : M	meter_reading.cycle_id	RESTRICT
billing_cycle	aggregated by	aggregation_run	1 : M	aggregation_run.cycle_id	RESTRICT
aggregation_run	writes	meter_reading	0..1 : M	meter_reading.run_id	SET NULL
property	receives	invoice	1 : M	invoice.property_id	RESTRICT
billing_cycle	bills	invoice	1 : M	invoice.cycle_id	RESTRICT
invoice	contains	invoice_line_item	1 : 1..M	invoice_line_item.invoice_id	CASCADE
utility_rate	prices	invoice_line_item	0..1 : M	invoice_line_item.rate_id	RESTRICT
invoice	paid via	payment	1 : M (≤ 1 SUCCEEDED)	payment.invoice_id	RESTRICT
property	raises	maintenance_ticket	1 : M	maintenance_ticket.property_id	RESTRICT
utility_meter	flagged in	maintenance_ticket	0..1 : M	maintenance_ticket.meter_id	RESTRICT
app_user	assigned	maintenance_ticket	0..1 : M	maintenance_ticket.assigned_to	SET NULL
maintenance_ticket	logs	ticket_status_history	1 : M	ticket_status_history.ticket_id	CASCADE
app_user	changed by	ticket_status_history	0..1 : M	ticket_status_history.changed_by	NO ACTION
app_user	has	user_session	1 : M	user_session.user_id	CASCADE

Why these ON DELETE rules. RESTRICT on money and ownership paths, because deleting a resident must never silently erase bills or payments (residents are soft-deleted instead). CASCADE only where the child cannot exist alone: an invoice's line items, a ticket's history, a user's sessions. SET NULL where the child should survive: a ticket keeps existing if its assigned staff account is removed.

5. Relational schema

Notation: **primary key** underlined, *foreign keys* in italics. All tables are in Third Normal Form. The only deliberate copies are `invoice.total_amount` and `invoice_line_item.rate_applied`, which are snapshots: a bill must keep the price that applied when it was issued, even if the tariff changes later. A trigger keeps the total equal to the sum of its lines.

zone

City → District → Ward hierarchy used for analytics and staff areas

`ZONE ( zone_id, name, zone_level, parent_zone_id )`

Column	Type	Key	Meaning / rule
zone_id	smallint	**PK**	
name	varchar	**UK**	
zone_level	varchar		CITY / DISTRICT / WARD
parent_zone_id	smallint	**FK**	→ zone, NULL only for CITY

Constraints

- CHECK zone_level IN ('CITY','DISTRICT','WARD')
- CHECK (zone_level = 'CITY') = (parent_zone_id IS NULL)

resident

A citizen who owns properties (synthetic data only)

`RESIDENT ( resident_id, citizen_ref, full_name, email, phone, registered_date, is_active )`

Column	Type	Key	Meaning / rule
resident_id	uuid	**PK**	
citizen_ref	varchar	**UK**	SYN-xxxxx, replaces government_id
full_name	varchar		
email	varchar	**UK**	lower-case
phone	varchar		
registered_date	date		
is_active	boolean		soft delete

Constraints

- NOT NULL on all columns
- CHECK citizen_ref LIKE 'SYN-%' (synthetic only)
- CHECK email is lower-case and contains '@'
- CHECK phone matches ^\+?[0-9]{10,14}$
- DEFAULT registered_date = CURRENT_DATE, is_active = TRUE

property

A house, shop or factory supplied with utilities

PROPERTY (**property_id**, property_code, *resident_id*, *zone_id*, address_line, city, pincode, property_type, square_feet, service_status)

Column	Type	Key	Meaning / rule
property_id	uuid	**PK**	
property_code	varchar	**UK**	P-0055
resident_id	uuid	**FK**	→ resident
zone_id	smallint	**FK**	→ zone
address_line	varchar		
city	varchar		
pincode	char		
property_type	varchar		RESIDENTIAL / COMMERCIAL / INDUSTRIAL
square_feet	int		
service_status	varchar		NORMAL / UNDER_INVESTIGATION / DISCONNECTED

Constraints

- CHECK property_type IN ('RESIDENTIAL','COMMERCIAL','INDUSTRIAL')
- CHECK square_feet BETWEEN 100 AND 1000000
- CHECK pincode is 6 digits
- CHECK service_status IN ('NORMAL','UNDER_INVESTIGATION','DISCONNECTED'), DEFAULT 'NORMAL'
- INDEX (resident_id), INDEX (zone_id)

utility_meter

A physical electricity, water or gas meter on a property

UTILITY_METER (**meter_id**, meter_code, *property_id*, utility_type, installation_date, meter_model, is_active)

Column	Type	Key	Meaning / rule
meter_id	uuid	**PK**	
meter_code	varchar	**UK**	MTR-E-000123, the id used in MongoDB
property_id	uuid	**FK**	→ property
utility_type	utility_kind (ENUM)		ELECTRICITY / WATER / GAS
installation_date	date		
meter_model	varchar		
is_active	boolean		

Constraints

- CHECK installation_date <= CURRENT_DATE
- Partial UNIQUE (property_id, utility_type) WHERE is_active: one active meter per utility per property

billing_cycle

One monthly billing period with its due date and penalty

```
BILLING_CYCLE ( cycle_id, year, month, cycle_start_date, cycle_end_date, due_date, late_penalty_percent, status )
```

Column	Type	Key	Meaning / rule
cycle_id	int	**PK**	
year	smallint		
month	smallint		1 to 12
cycle_start_date	date		
cycle_end_date	date		
due_date	date		
late_penalty_percent	numeric		
status	varchar		OPEN / BILLING / CLOSED

Constraints

- UNIQUE (year, month)
- CHECK month BETWEEN 1 AND 12, year BETWEEN 2020 AND 2100
- CHECK cycle_start_date < cycle_end_date < due_date
- CHECK late_penalty_percent BETWEEN 0 AND 25, DEFAULT 5.00
- CHECK status IN ('OPEN','BILLING','CLOSED')

utility_rate

One tariff slab (tier) for a utility, valid for a date range

```
UTILITY_RATE ( rate_id, utility_type, tier_level, min_units, max_units, price_per_unit, effective_start_date, effective_end_date )
```

Column	Type	Key	Meaning / rule
rate_id	int	**PK**	
utility_type	utility_kind (ENUM)		
tier_level	smallint		
min_units	numeric		
max_units	numeric		NULL = no upper limit
price_per_unit	numeric		
effective_start_date	date		
effective_end_date	date		

Constraints

- CHECK min_units >= 0, price_per_unit > 0
- CHECK max_units IS NULL OR max_units > min_units
- CHECK effective_end_date IS NULL OR > effective_start_date
- EXCLUDE USING gist (utility_type =, tier_level =, daterange(start, end) &&): no overlapping versions of a slab
- Constraint trigger: slabs start at 0, have no gaps, only the top tier is unbounded

aggregation_run

One run of the MongoDB → PostgreSQL monthly usage bridge

```
AGGREGATION_RUN ( run_id, cycle_id, started_at, finished_at, status, meters_total, meters_written, error )
```

Column	Type	Key	Meaning / rule
run_id	uuid	**PK**	
cycle_id	int	**FK**	→ billing_cycle
started_at	timestamptz		
finished_at	timestamptz		
status	varchar		RUNNING / SUCCEEDED / FAILED
meters_total	int		
meters_written	int		
error	text		

Constraints

- CHECK status IN ('RUNNING','SUCCEEDED','FAILED')
- Partial UNIQUE (cycle_id) WHERE status = 'RUNNING': one running bridge per cycle

meter_reading

Monthly consumption of one meter, aggregated from MongoDB (the bridge table)

```
METER_READING ( reading_id, meter_id, cycle_id, run_id, opening_register, closing_register, total_units_consumed, source_doc_count, is_estimated, reading_timestamp )
```

Column	Type	Key	Meaning / rule
reading_id	uuid	**PK**	
meter_id	uuid	**FK**	→ utility_meter
cycle_id	int	**FK**	→ billing_cycle
run_id	uuid	**FK**	→ aggregation_run
opening_register	numeric		
closing_register	numeric		
total_units_consumed	numeric		generated: closing − opening
source_doc_count	int		
is_estimated	boolean		
reading_timestamp	timestamptz		

Constraints

- UNIQUE (meter_id, cycle_id): one monthly reading per meter
- CHECK opening_register >= 0, closing_register >= opening_register
- total_units_consumed GENERATED ALWAYS AS (closing − opening) STORED
- CHECK source_doc_count >= 0
- Trigger: no UPDATE/DELETE once the cycle is invoiced

invoice

The bill for one property in one billing cycle

INVOICE (**invoice_id**, invoice_no, *property_id*, *cycle_id*, invoice_date, total_amount, payment_status, paid_at)

Column	Type	Key	Meaning / rule
invoice_id	uuid	**PK**	
invoice_no	varchar	**UK**	
property_id	uuid	**FK**	→ property
cycle_id	int	**FK**	→ billing_cycle
invoice_date	date		
total_amount	numeric		kept equal to sum of lines by trigger
payment_status	varchar		PENDING / PAID / OVERDUE / DISPUTED
paid_at	timestamptz		

Constraints

- UNIQUE (property_id, cycle_id): no duplicate bills
- CHECK total_amount >= 0
- CHECK payment_status IN ('PENDING','PAID','OVERDUE','DISPUTED'), DEFAULT 'PENDING'
- CHECK (payment_status = 'PAID') = (paid_at IS NOT NULL)
- INDEX (payment_status, cycle_id)

invoice_line_item

One line of a bill: a utility × tier, or a late fee

INVOICE_LINE_ITEM (**line_item_id**, *invoice_id*, utility_type, line_kind, *rate_id*, tier_level, units_consumed, rate_applied, line_total)

Column	Type	Key	Meaning / rule
line_item_id	uuid	**PK**	
invoice_id	uuid	**FK**	→ invoice
utility_type	utility_kind (ENUM)		
line_kind	varchar		USAGE / LATE_FEE
rate_id	int	**FK**	→ utility_rate
tier_level	smallint		
units_consumed	numeric		
rate_applied	numeric		price snapshot
line_total	numeric		

Constraints

- UNIQUE (invoice_id, utility_type, line_kind, tier_level)
- CHECK units_consumed, rate_applied, line_total >= 0
- CHECK line_kind = 'LATE_FEE' OR (rate_id and tier_level NOT NULL)
- Trigger: keeps invoice.total_amount = SUM(line_total); blocks edits on PAID invoices

payment

Every payment attempt (simulated gateway); at most one succeeds per invoice

PAYMENT (**payment_id**, *invoice_id*, idempotency_key, amount_paid, payment_method, status, transaction_id, failure_reason, payment_date)

Column	Type	Key	Meaning / rule
payment_id	uuid	**PK**	
invoice_id	uuid	**FK**	→ invoice
idempotency_key	varchar	**UK**	blocks replayed requests
amount_paid	numeric		
payment_method	varchar		UPI_SIM / CARD_SIM / NETBANK_SIM / CASH
status	varchar		INITIATED / SUCCEEDED / FAILED
transaction_id	varchar	**UK**	
failure_reason	text		
payment_date	timestamptz		

Constraints

- Partial UNIQUE (invoice_id) WHERE status = 'SUCCEEDED': one final payment per invoice
- CHECK amount_paid > 0
- CHECK payment_method IN ('UPI_SIM','CARD_SIM','NETBANK_SIM','CASH')
- CHECK status IN ('INITIATED','SUCCEEDED','FAILED')
- Trigger: SUCCEEDED payment marks the invoice PAID

maintenance_ticket

A repair job, opened by a resident, staff or the anomaly detector

MAINTENANCE_TICKET (**ticket_id**, ticket_no, *property_id*, *meter_id*, source, anomaly_key, issue_type, severity, description, status, *assigned_to*, reported_timestamp, last_seen_at, occurrence_count, resolved_at, resolution_notes)

Column	Type	Key	Meaning / rule
ticket_id	uuid	**PK**	
ticket_no	varchar	**UK**	
property_id	uuid	**FK**	→ property
meter_id	uuid	**FK**	→ utility_meter
source	varchar		ANOMALY / RESIDENT / STAFF
anomaly_key	varchar		one active ticket per key
issue_type	varchar		
severity	varchar		LOW / MEDIUM / HIGH / CRITICAL
description	text		
status	varchar		OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED
assigned_to	uuid	**FK**	→ app_user
reported_timestamp	timestamptz		
last_seen_at	timestamptz		
occurrence_count	int		
resolved_at	timestamptz		
resolution_notes	text		

Constraints

- Partial UNIQUE (anomaly_key) WHERE status IN ('OPEN','ASSIGNED','IN_PROGRESS'): no duplicate live tickets
- CHECK issue_type, severity, source, status in allowed lists
- CHECK length(description) BETWEEN 5 AND 2000, occurrence_count >= 1
- CHECK RESOLVED/CLOSED requires resolved_at and resolution_notes
- Trigger: only legal status transitions; writes ticket_status_history
- INDEX (status, severity), partial INDEX (assigned_to)

ticket_status_history

Audit trail of every ticket status change (written by trigger)

TICKET_STATUS_HISTORY (**history_id**, *ticket_id*, from_status, to_status, *changed_by*, changed_at)

Column	Type	Key	Meaning / rule
history_id	bigserial	**PK**	
ticket_id	uuid	**FK**	→ maintenance_ticket
from_status	varchar		
to_status	varchar		
changed_by	uuid	**FK**	→ app_user
changed_at	timestamptz		

Constraints

- NOT NULL ticket_id, to_status, changed_at (DEFAULT now())

app_user

A login account with its role (Resident, Staff or Admin)

`APP_USER ( user_id, email, password_hash, role, resident_id, staff_zone_id, failed_logins, locked_until )`

Column	Type	Key	Meaning / rule
user_id	uuid	**PK**	
email	varchar	**UK**	
password_hash	text		Argon2id
role	varchar		RESIDENT / STAFF / ADMIN
resident_id	uuid	**FK, UK**	→ resident (residents only)
staff_zone_id	smallint	**FK**	→ zone (staff only)
failed_logins	smallint		
locked_until	timestamptz		

Constraints

- CHECK role IN ('RESIDENT','STAFF','ADMIN')
- CHECK (role = 'RESIDENT') = (resident_id IS NOT NULL)
- CHECK role <> 'STAFF' OR staff_zone_id IS NOT NULL

user_session

A signed-in session; only a hash of the cookie token is stored

`USER_SESSION ( token_hash, user_id, created_at, expires_at )`

Column	Type	Key	Meaning / rule
token_hash	bytea	**PK**	sha256 of token
user_id	uuid	**FK**	→ app_user
created_at	timestamptz		
expires_at	timestamptz		

Constraints

- CHECK expires_at > created_at

Appendix: PostgreSQL DDL (draft)

Draft from the plan; it becomes the migration files `0001` and `0002` on Days 1–2 and will be tested against a real PostgreSQL 17 database there. Triggers and functions named above are in later migrations.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS btree_gist;  -- EXCLUDE with = and &&

CREATE TYPE utility_kind AS ENUM ('ELECTRICITY','WATER','GAS');

CREATE TABLE zone (
  zone_id        SMALLSERIAL PRIMARY KEY,
  name           VARCHAR(60) NOT NULL UNIQUE,
  zone_level     VARCHAR(10) NOT NULL CHECK (zone_level IN ('CITY','DISTRICT','WARD')),
  parent_zone_id SMALLINT REFERENCES zone(zone_id) ON DELETE RESTRICT,
  CHECK ((zone_level = 'CITY') = (parent_zone_id IS NULL))
);

CREATE TABLE resident (
  resident_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  citizen_ref     VARCHAR(20) NOT NULL UNIQUE CHECK (citizen_ref LIKE 'SYN-%'),  -- synthetic, replaces government_id
  full_name       VARCHAR(100) NOT NULL,
  email           VARCHAR(254) NOT NULL UNIQUE CHECK (email = lower(email) AND position('@' in email) > 1),
  phone           VARCHAR(15) NOT NULL CHECK (phone ~ '^\+?[0-9]{10,14}$'),
  registered_date DATE NOT NULL DEFAULT CURRENT_DATE,
  is_active       BOOLEAN NOT NULL DEFAULT TRUE              -- soft delete
);
```

```
CREATE TABLE property (
  property_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_code  VARCHAR(12) NOT NULL UNIQUE,               -- human id, e.g. P-0055
  resident_id    UUID NOT NULL REFERENCES resident(resident_id) ON DELETE RESTRICT,
  zone_id        SMALLINT NOT NULL REFERENCES zone(zone_id) ON DELETE RESTRICT,
  address_line   VARCHAR(255) NOT NULL,
  city           VARCHAR(50)  NOT NULL DEFAULT 'Synthetic City',
  pincode        CHAR(6)      NOT NULL CHECK (pincode ~ '^[1-9][0-9]{5}$'),
  property_type  VARCHAR(12)  NOT NULL CHECK (property_type IN ('RESIDENTIAL','COMMERCIAL','INDUSTRIAL')),
  square_feet    INT          NOT NULL CHECK (square_feet BETWEEN 100 AND 1000000),
  service_status VARCHAR(20)  NOT NULL DEFAULT 'NORMAL'
                 CHECK (service_status IN ('NORMAL','UNDER_INVESTIGATION','DISCONNECTED'))
);
CREATE INDEX ON property (resident_id);
CREATE INDEX ON property (zone_id);

CREATE TABLE utility_meter (
  meter_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  meter_code        VARCHAR(16) NOT NULL UNIQUE,             -- e.g. MTR-E-000123 (used in Mongo)
  property_id       UUID NOT NULL REFERENCES property(property_id) ON DELETE RESTRICT,
  utility_type      utility_kind NOT NULL,
  installation_date DATE NOT NULL CHECK (installation_date <= CURRENT_DATE),
  meter_model       VARCHAR(50),
  is_active         BOOLEAN NOT NULL DEFAULT TRUE
);
-- one active meter per utility per property
CREATE UNIQUE INDEX uq_active_meter ON utility_meter (property_id, utility_type) WHERE is_active;

CREATE TABLE billing_cycle (
  cycle_id             SERIAL PRIMARY KEY,
  year                 SMALLINT NOT NULL CHECK (year BETWEEN 2020 AND 2100),
  month                SMALLINT NOT NULL CHECK (month BETWEEN 1 AND 12),
  cycle_start_date     DATE NOT NULL,
  cycle_end_date       DATE NOT NULL,
  due_date             DATE NOT NULL,
  late_penalty_percent NUMERIC(5,2) NOT NULL DEFAULT 5.00 CHECK (late_penalty_percent BETWEEN 0 AND 25),
  status               VARCHAR(10) NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','BILLING','CLOSED')),
  UNIQUE (year, month),
  CHECK (cycle_start_date < cycle_end_date AND cycle_end_date < due_date)
);

CREATE TABLE utility_rate (
  rate_id              SERIAL PRIMARY KEY,
  utility_type         utility_kind NOT NULL,
  tier_level           SMALLINT NOT NULL CHECK (tier_level BETWEEN 1 AND 5),
  min_units            NUMERIC(12,3) NOT NULL CHECK (min_units >= 0),
  max_units            NUMERIC(12,3) CHECK (max_units IS NULL OR max_units > min_units),  -- NULL = no upper limit
  price_per_unit       NUMERIC(10,4) NOT NULL CHECK (price_per_unit > 0),
  effective_start_date DATE NOT NULL,
  effective_end_date   DATE CHECK (effective_end_date IS NULL OR effective_end_date > effective_start_date),
  -- no two versions of the same slab may be valid on the same day
  EXCLUDE USING gist (utility_type WITH =, tier_level WITH =,
                      daterange(effective_start_date, effective_end_date, '[)') WITH &&)
);
-- plus constraint trigger T4 (contiguous slabs, tier 1 starts at 0, only top tier unbounded)

CREATE TABLE aggregation_run (
  run_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cycle_id      INT NOT NULL REFERENCES billing_cycle(cycle_id) ON DELETE RESTRICT,
  started_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  finished_at   TIMESTAMPTZ,
  status        VARCHAR(10) NOT NULL DEFAULT 'RUNNING' CHECK (status IN ('RUNNING','SUCCEEDED','FAILED')),
  meters_total  INT, meters_written INT, error TEXT
);
CREATE UNIQUE INDEX uq_one_running_run ON aggregation_run (cycle_id) WHERE status = 'RUNNING';

CREATE TABLE meter_reading (
  reading_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  meter_id             UUID NOT NULL REFERENCES utility_meter(meter_id) ON DELETE RESTRICT,
  cycle_id             INT  NOT NULL REFERENCES billing_cycle(cycle_id) ON DELETE RESTRICT,
  run_id               UUID REFERENCES aggregation_run(run_id) ON DELETE SET NULL,
  opening_register     NUMERIC(15,3) NOT NULL CHECK (opening_register >= 0),
  closing_register     NUMERIC(15,3) NOT NULL,
  total_units_consumed NUMERIC(15,3) GENERATED ALWAYS AS (closing_register - opening_register) STORED,
  source_doc_count     INT NOT NULL CHECK (source_doc_count >= 0),
  is_estimated         BOOLEAN NOT NULL DEFAULT FALSE,      -- true when data gaps were filled
  reading_timestamp    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (meter_id, cycle_id),
  CHECK (closing_register >= opening_register)
);

CREATE TABLE invoice (
  invoice_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  invoice_no     VARCHAR(20) NOT NULL UNIQUE,              -- INV-2026-09-000123
  property_id    UUID NOT NULL REFERENCES property(property_id) ON DELETE RESTRICT,
  cycle_id       INT  NOT NULL REFERENCES billing_cycle(cycle_id) ON DELETE RESTRICT,
  invoice_date   DATE NOT NULL DEFAULT CURRENT_DATE,
  total_amount   NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total_amount >= 0),
  payment_status VARCHAR(10) NOT NULL DEFAULT 'PENDING'
                 CHECK (payment_status IN ('PENDING','PAID','OVERDUE','DISPUTED')),
  paid_at        TIMESTAMPTZ,
  UNIQUE (property_id, cycle_id),                          -- no duplicate bills
  CHECK ((payment_status = 'PAID') = (paid_at IS NOT NULL))
);
CREATE INDEX ON invoice (payment_status, cycle_id);

CREATE TABLE invoice_line_item (
  line_item_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id     UUID NOT NULL REFERENCES invoice(invoice_id) ON DELETE CASCADE,
  utility_type   utility_kind NOT NULL,
  line_kind      VARCHAR(10) NOT NULL DEFAULT 'USAGE' CHECK (line_kind IN ('USAGE','LATE_FEE')),
  rate_id        INT REFERENCES utility_rate(rate_id) ON DELETE RESTRICT,
  tier_level     SMALLINT,
  units_consumed NUMERIC(15,3) NOT NULL CHECK (units_consumed >= 0),
  rate_applied   NUMERIC(10,4) NOT NULL CHECK (rate_applied >= 0),     -- price snapshot
  line_total     NUMERIC(12,2) NOT NULL CHECK (line_total >= 0),
  UNIQUE (invoice_id, utility_type, line_kind, tier_level),
  CHECK (line_kind = 'LATE_FEE' OR (rate_id IS NOT NULL AND tier_level IS NOT NULL))
);

CREATE TABLE payment (
  payment_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id      UUID NOT NULL REFERENCES invoice(invoice_id) ON DELETE RESTRICT,
  idempotency_key VARCHAR(64) NOT NULL UNIQUE,              -- replayed requests hit this
  amount_paid     NUMERIC(12,2) NOT NULL CHECK (amount_paid > 0),
  payment_method  VARCHAR(12) NOT NULL CHECK (payment_method IN ('UPI_SIM','CARD_SIM','NETBANK_SIM','CASH')),
  status          VARCHAR(10) NOT NULL DEFAULT 'INITIATED' CHECK (status IN ('INITIATED','SUCCEEDED','FAILED')),
  transaction_id  VARCHAR(100) UNIQUE,                      -- simulated gateway reference
  failure_reason  TEXT,
  payment_date    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_one_success_per_invoice ON payment (invoice_id) WHERE status = 'SUCCEEDED';

CREATE TABLE app_user (
  user_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         VARCHAR(254) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,                              -- Argon2id encoded string
  role          VARCHAR(10) NOT NULL CHECK (role IN ('RESIDENT','STAFF','ADMIN')),
  resident_id   UUID UNIQUE REFERENCES resident(resident_id) ON DELETE RESTRICT,
  staff_zone_id SMALLINT REFERENCES zone(zone_id),
  failed_logins SMALLINT NOT NULL DEFAULT 0, locked_until TIMESTAMPTZ,
  CHECK ((role = 'RESIDENT') = (resident_id IS NOT NULL)),
  CHECK (role <> 'STAFF' OR staff_zone_id IS NOT NULL)
);

CREATE TABLE user_session (
  token_hash  BYTEA PRIMARY KEY,                            -- sha256(token); raw token only in cookie
  user_id     UUID NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at  TIMESTAMPTZ NOT NULL,
  CHECK (expires_at > created_at)
);

CREATE TABLE maintenance_ticket (
  ticket_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_no          VARCHAR(12) NOT NULL UNIQUE,           -- MT-000221
  property_id        UUID NOT NULL REFERENCES property(property_id) ON DELETE RESTRICT,
  meter_id           UUID REFERENCES utility_meter(meter_id) ON DELETE RESTRICT,
  source             VARCHAR(10) NOT NULL CHECK (source IN ('ANOMALY','RESIDENT','STAFF')),
  anomaly_key        VARCHAR(80),                           -- e.g. LEAK:MTR-W-000012
  issue_type         VARCHAR(20) NOT NULL CHECK (issue_type IN
                       ('LEAK','POWER_OUTAGE','VOLTAGE_SPIKE','METER_MALFUNCTION','LOW_PRESSURE','BILLING','OTHER')),
  severity           VARCHAR(8) NOT NULL CHECK (severity IN ('LOW','MEDIUM','HIGH','CRITICAL')),
  description        TEXT NOT NULL CHECK (length(description) BETWEEN 5 AND 2000),
  status             VARCHAR(12) NOT NULL DEFAULT 'OPEN'
                     CHECK (status IN ('OPEN','ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED')),
  assigned_to        UUID REFERENCES app_user(user_id) ON DELETE SET NULL,
  reported_timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_seen_at       TIMESTAMPTZ NOT NULL DEFAULT now(),       -- updated while anomaly continues
  occurrence_count   INT NOT NULL DEFAULT 1 CHECK (occurrence_count >= 1),
  resolved_at        TIMESTAMPTZ,
  resolution_notes   TEXT,
  CHECK (status NOT IN ('RESOLVED','CLOSED') OR (resolved_at IS NOT NULL AND resolution_notes IS NOT NULL))
);
-- the duplicate-ticket guard: one *active* ticket per continuing anomaly
CREATE UNIQUE INDEX uq_active_anomaly_ticket ON maintenance_ticket (anomaly_key)
  WHERE status IN ('OPEN','ASSIGNED','IN_PROGRESS');
```

```
CREATE INDEX ON maintenance_ticket (status, severity);
CREATE INDEX ON maintenance_ticket (assigned_to) WHERE status IN ('ASSIGNED','IN_PROGRESS');

CREATE TABLE ticket_status_history (
  history_id  BIGSERIAL PRIMARY KEY,
  ticket_id   UUID NOT NULL REFERENCES maintenance_ticket(ticket_id) ON DELETE CASCADE,
  from_status VARCHAR(12), to_status VARCHAR(12) NOT NULL,
  changed_by  UUID REFERENCES app_user(user_id),
  changed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```